

读享 V1.0 软件

后 30 页

著作权人:XX 有限公司

生成日期:2026-06-15

摘选总行数:1500

文件清单:

- person/notfresh/readingshare/core/Synchronizer.java
- person/notfresh/readingshare/util/PdfOutlineExtractor.java
- person/notfresh/readingshare/util/BilibiliUrlConverter.java
- person/notfresh/readingshare/util/CrawlUtil.java
- person/notfresh/readingshare/util/ExportUtil.java
- person/notfresh/readingshare/util/ImportUtil.java

第2页 / 共33页

100

```

101         linkObj.put("title", link.getTitle());
102         linkObj.put("url", link.getUrl());
103         linkObj.put("timestamp", isoFormat.format(new Date(link.getTimestamp
104         linksArray.put(linkObj);
105     }
106
107     jsonData.put("links", linksArray);
108     return jsonData;
109 }
110
111 /**
112  * ■■■HTTP■■■
113  */
114 private SyncResult sendRequest(String targetUrl, JSONObject jsonData) {
115     try {
116         URL url = new URL(targetUrl);
117         HttpURLConnection conn = (HttpURLConnection) url.openConnection();
118
119         // TODO ■■■SSL■■■■■■■■■■
120         //configureSSL(conn);
121
122         // ■■■■■■■■■■
123         conn.setRequestMethod("POST");
124         conn.setRequestProperty("Content-Type", "application/json");
125         conn.setDoOutput(true);
126
127         Log.d(TAG, "■■■■■■■■: " + targetUrl);
128
129         // ■■■■■■
130         try (OutputStream os = conn.getOutputStream()) {
131             byte[] input = jsonData.toString().getBytes(StandardCharsets.UTF
132             os.write(input, 0, input.length);
133         }
134
135         // ■■■■
136         int responseCode = conn.getResponseCode();
137         Log.d(TAG, "■■■■■■■■: " + responseCode);
138
139         if (responseCode == HttpURLConnection.HTTP_OK) {
140             // ■■■■■■
141             String responseData = readResponse(conn);
142             return SyncResult.success(responseData, responseCode);
143         } else {
144             // ■■■■■■
145             String errorResponse = readErrorResponse(conn);
146             return SyncResult.failure("■■■■■■■■: " + responseCode +
147                 (errorResponse != null ? " - " + errorResponse : ""), respon
148         }
149     } catch (IOException e) {

```



```
201     }
202 }
203
204 /**
205  * ████████
206  */
207 private String readErrorResponse(HttpURLConnection conn) {
208     try {
209         if (conn.getErrorStream() != null) {
210             try (BufferedReader reader = new BufferedReader(
211                 new InputStreamReader(conn.getErrorStream(), StandardCha
212                     StringBuilder response = new StringBuilder();
213                     String line;
214                     while ((line = reader.readLine()) != null) {
215                         response.append(line);
216                     }
217                     return response.toString();
218                 }
219             }
220         } catch (IOException e) {
221             Log.e(TAG, "██████████", e);
222         }
223         return null;
224     }
225 }
226
```

50

100


```

101                 count++;
102
103                 Log.d(TAG, "■■■■■: " + title + " -> ■" + (pageIndex
104                 }
105             }
106         } catch (Exception e) {
107             Log.w(TAG, "■■■■■■■", e);
108         }
109     }
110
111     // ■■2■■■■■1■■■■■■■■■■ /Title ■■
112     if (outlineItems.isEmpty()) {
113         extractOutlinesSimple(pdfContent, outlineItems);
114     }
115
116     // ■■■■■
117     deduplicateAndSort(outlineItems);
118
119     } catch (Exception e) {
120         Log.e(TAG, "■■PDF■■■■■", e);
121     }
122 }
123
124 /**
125  * ■■■■■■■■■■■■
126  */
127 private static boolean isValidOutlineTitle(String title) {
128     if (title == null || title.trim().isEmpty()) {
129         return false;
130     }
131
132     // ■■■■■■■■■■■■
133     String lowerTitle = title.toLowerCase();
134     String[] invalidTitles = {
135         "metadata", "info", "xref", "trailer", "catalog",
136         "root", "pages", "page", "outlines", "acroform"
137     };
138
139     for (String invalid : invalidTitles) {
140         if (lowerTitle.equals(invalid)) {
141             return false;
142         }
143     }
144
145     // ■■■■■■■■■■■■
146     return title.length() >= 1 && title.length() <= 200;
147 }
148
149 /**
150  * ■■■■■■■■■■■■

```

```

151 * ██████████Parent/First████████
152 */
153 private static int estimateLevel(int position, String pdfContent) {
154     try {
155         // ██████████Parent██
156         int start = Math.max(0, position - 1000);
157         int end = Math.min(pdfContent.length(), position + 1000);
158         String context = pdfContent.substring(start, position);
159
160         // ██Parent██████████Parent██████████
161         Pattern parentPattern = Pattern.compile("/Parent");
162         Matcher parentMatcher = parentPattern.matcher(context);
163         int parentCount = 0;
164         while (parentMatcher.find()) {
165             parentCount++;
166         }
167
168         return Math.min(parentCount, 5); // █████5██
169     } catch (Exception e) {
170         return 0;
171     }
172 }
173
174 /**
175  * ██████████
176  */
177 private static void deduplicateAndSort(List<TocOutlineItem> items) {
178     if (items.isEmpty()) {
179         return;
180     }
181
182     // ████████
183     items.sort((a, b) -> Integer.compare(a.getPageIndex(), b.getPageIndex()));
184
185     // ████████████████████████████████████████
186     List<TocOutlineItem> unique = new ArrayList<>();
187     TocOutlineItem last = null;
188
189     for (TocOutlineItem item : items) {
190         if (last == null ||
191             !item.getTitle().equals(last.getTitle()) ||
192             item.getPageIndex() != last.getPageIndex()) {
193             unique.add(item);
194             last = item;
195         }
196     }
197
198     items.clear();
199     items.addAll(unique);
200 }

```

第 11 页 / 共 33 页

第 13 页 / 共 33 页

第 14 页 / 共 33 页

第 15 页 / 共 33 页

```

451     private static int findPageInContext(String context) {
452         // ■■ /Dest [■■ ■ /Page ■■
453         Pattern destPattern = Pattern.compile("/Dest\\s*\\[\\s*(\\d+)");
454         Matcher destMatcher = destPattern.matcher(context);
455         if (destMatcher.find()) {
456             try {
457                 int page = Integer.parseInt(destMatcher.group(1));
458                 return Math.max(0, page - 1);
459             } catch (NumberFormatException ignored) {
460             }
461         }
462
463         // ■■ /Page (■■)
464         Pattern pagePattern = Pattern.compile("/Page\\s*\\((\\d+)");
465         Matcher pageMatcher = pagePattern.matcher(context);
466         if (pageMatcher.find()) {
467             try {
468                 int page = Integer.parseInt(pageMatcher.group(1));
469                 return Math.max(0, page - 1);
470             } catch (NumberFormatException ignored) {
471             }
472         }
473
474         return -1;
475     }
476 }
477

```



```
1 package person.notfresh.readingshare.util;
2
3 import java.io.IOException;
4 import java.net.HttpURLConnection;
5 import java.net.URL;
6
7 public class BilibiliUrlConverter {
8
9     public static String getRedirectedUrl(String shortUrl) throws IOException {
10         HttpURLConnection connection = (HttpURLConnection) new URL(shortUrl).openConnection();
11         connection.setInstanceFollowRedirects(false);
12         connection.connect();
13         String redirectedUrl = connection.getHeaderField("Location");
14         connection.disconnect();
15         return redirectedUrl;
16     }
17
18     public static String convertToBilibiliScheme(String url) {
19         if (url == null) {
20             return null;
21         }
22         if (url.startsWith("https://www.bilibili.com/video/")) {
23             return url.replace("https://www.bilibili.com/video/", "bilibili://video/");
24         }
25         return url;
26     }
27 }
```


100

[illegible]

```
151         }
152         public void checkClientTrusted(java.security.cert.X509Certif
153         }
154         public void checkServerTrusted(java.security.cert.X509Certif
155         }
156     }
157 };
158
159     javax.net.ssl.SSLContext sc = javax.net.ssl.SSLContext.getInstance("
160     sc.init(null, trustAllCerts, new java.security.SecureRandom());
161     httpsConn.setSSLSocketFactory(sc.getSocketFactory());
162     httpsConn.setHostnameVerifier((hostname, session) -> true);
163 }
164 conn.setRequestMethod("POST");
165 conn.setConnectTimeout(Config.Network.CONNECT_TIMEOUT);
166 conn.setReadTimeout(timeoutSeconds * 1000);
167 conn.setDoOutput(true);
168 conn.setRequestProperty("Content-Type", Config.Headers.CONTENT_TYPE_JSON
169
170 // ■■■■JSON
171 JSONObject requestJson = new JSONObject();
172 requestJson.put("url", url);
173 requestJson.put("key", Config.API.DUXIANG_API_KEY);
174
175 // ■■■■■■
176 try (java.io.OutputStream os = conn.getOutputStream()) {
177     byte[] input = requestJson.toString().getBytes("utf-8");
178     os.write(input, 0, input.length);
179 }
180
181 // ■■■■
182 StringBuilder response = new StringBuilder();
183 try (BufferedReader reader = new BufferedReader(
184     new InputStreamReader(conn.getInputStream(), StandardCharsets.UTF_8)
185     String line;
186     while ((line = reader.readLine()) != null) {
187         response.append(line);
188     }
189 }
190
191 // ■■JSON■■
192 JSONObject jsonResponse = new JSONObject(response.toString());
193 if (jsonResponse.getBoolean("success")) {
194     JSONObject data = jsonResponse.getJSONObject("data");
195     return data.getString("summary");
196 }
197
198 throw new IOException("Failed to get summary from API");
199 }
200
```

第 22 页 / 共 33 页

```

251         conn.setRequestProperty("Accept", "text/html,application/xhtml+xml,a
252
253         // ■■■■ HTML■■■■■ </head> ■■
254         StringBuilder html = new StringBuilder();
255         try (BufferedReader reader = new BufferedReader(
256             new InputStreamReader(conn.getInputStream(), StandardCharset
257             String line;
258             int linesRead = 0;
259             final int MAX_LINES = 500; // ■■■■ 500 ■
260             final int MAX_SIZE = 50 * 1024; // ■■ 50KB
261
262             while ((line = reader.readLine()) != null && linesRead < MAX_LIN
263                 html.append(line).append("\n");
264                 linesRead++;
265
266                 // ■■■■■■ </head> ■■■■■■■■■■
267                 String htmlStr = html.toString().toLowerCase();
268                 if (htmlStr.contains("</head>")) {
269                     Log.d("CrawlUtil", "Found </head> tag, stopping at line
270                     break;
271                 }
272
273                 // ■■■■■■■■
274                 if (html.length() > MAX_SIZE) {
275                     Log.d("CrawlUtil", "Reached max size limit, stopping");
276                     break;
277                 }
278             }
279         }
280
281         // ■■ Jsoup ■■ HTML ■■■■■■ head ■■■
282         Document doc = Jsoup.parse(html.toString());
283
284         // ■■■■■■ favicon ■■
285         String faviconUrl = null;
286
287         // 1. ■■ <link rel="icon">
288         Elements iconLinks = doc.select("link[rel=icon]");
289         if (!iconLinks.isEmpty()) {
290             faviconUrl = iconLinks.first().attr("href");
291             Log.d("CrawlUtil", "Found favicon via rel=icon: " + faviconUrl);
292         }
293
294         // 2. ■■■■■■■■ <link rel="shortcut icon">
295         if (faviconUrl == null || faviconUrl.isEmpty()) {
296             Elements shortcutIconLinks = doc.select("link[rel=shortcut icon]
297             if (!shortcutIconLinks.isEmpty()) {
298                 faviconUrl = shortcutIconLinks.first().attr("href");
299                 Log.d("CrawlUtil", "Found favicon via rel=shortcut icon: " +
300             }

```

第24页 / 共33页

第 25 页 / 共 33 页

449 }

```

1 package person.notfresh.readingshare.util;
2
3 import android.content.ContentResolver;
4 import android.content.ContentValues;
5 import android.content.Context;
6 import android.net.Uri;
7 import android.os.Build;
8 import android.os.Environment;
9 import android.provider.MediaStore;
10 import android.text.TextUtils;
11 import org.json.JSONArray;
12 import org.json.JSONObject;
13 import java.io.File;
14 import java.io.FileOutputStream;
15 import java.io.IOException;
16 import java.io.OutputStream;
17 import java.io.OutputStreamWriter;
18 import java.nio.charset.StandardCharsets;
19 import java.text.SimpleDateFormat;
20 import java.util.Date;
21 import java.util.List;
22 import java.util.Locale;
23 import person.notfresh.readingshare.model.LinkItem;
24 import android.util.Log;
25 import org.json.JSONException;
26
27 public class ExportUtil {
28     /**
29      * ■■■JSON■■■■■■■■■■■
30      */
31     public static String exportToJson(Context context, List<LinkItem> links, Str
        JSONArray jsonArray = new JSONArray();
32
33         for (LinkItem link : links) {
34             try {
35                 JSONObject jsonObject = new JSONObject();
36                 jsonObject.put("title", link.getTitle());
37                 jsonObject.put("url", link.getUrl());
38                 jsonObject.put("tags", new JSONArray(link.getTags()));
39                 jsonArray.put(jsonObject);
40             } catch (JSONException e) {
41                 Log.e("ExportUtil", "Error creating JSON object", e);
42             }
43         }
44
45         // ■■■■■.json■■■
46         if (!fileName.toLowerCase().endsWith(".json")) {
47             fileName += ".json";
48         }

```

```

51
52     File exportDir = new File(context.getExternalFilesDir(null), "exports");
53     if (!exportDir.exists()) {
54         exportDir.mkdirs();
55     }
56
57     File file = new File(exportDir, fileName);
58     // UTF-8
59     OutputStreamWriter writer = new OutputStreamWriter(
60         new FileOutputStream(file), StandardCharsets.UTF_8);
61     // JSONObject \ / -> / JSON
62     String jsonString = jsonArray.toString(4).replace("\\/", "/");
63     writer.write(jsonString);
64     writer.flush();
65     writer.close();
66
67     return file.getAbsolutePath();
68 }
69
70 /**
71  * CSV
72  */
73 public static String exportToCsv(Context context, List<LinkItem> links, Stri
74     // .csv
75     if (!fileName.toLowerCase().endsWith(".csv")) {
76         fileName += ".csv";
77     }
78
79     File exportDir = new File(context.getExternalFilesDir(null), "exports");
80     if (!exportDir.exists()) {
81         exportDir.mkdirs();
82     }
83
84     File file = new File(exportDir, fileName);
85     // UTF-8
86     OutputStreamWriter writer = new OutputStreamWriter(
87         new FileOutputStream(file), StandardCharsets.UTF_8);
88
89     // CSV
90     StringBuilder csv = new StringBuilder();
91     // CSV
92     csv.append(",,,\n");
93
94     SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Local
95
96     //
97     for (LinkItem link : links) {
98         String title = escapeCSV(link.getTitle());
99         String url = escapeCSV(link.getUrl());
100         String date = sdf.format(new Date(link.getTimestamp()));

```

第 29 页 / 共 33 页

[illegible]

```

201         if (!documentsFolder.exists()) {
202             documentsFolder.mkdirs();
203         }
204
205         File outputFile = new File(documentsFolder, fileName);
206         FileOutputStream fos = new FileOutputStream(outputFile);
207         writeDataToStream(fos, links, isJson);
208         fos.close();
209
210         return Uri.fromFile(outputFile);
211     }
212 }
213
214 /**
215  * ■■■■■■■■■■
216  * @param outputStream ■■■
217  * @param links ■■■■
218  * @param isJson ■■■ JSON ■■
219  * @throws IOException ■■■■■■
220  * @throws JSONException JSON ■■■■
221  */
222 private static void writeDataToStream(OutputStream outputStream, List<LinkIt
223                                     boolean isJson) throws IOException, JSO
224     OutputStreamWriter writer = new OutputStreamWriter(
225         outputStream, StandardCharsets.UTF_8);
226
227     if (isJson) {
228         // ■■ JSON ■■
229         JSONArray jsonArray = new JSONArray();
230         for (LinkItem link : links) {
231             try {
232                 JSONObject jsonObject = new JSONObject();
233                 jsonObject.put("title", link.getTitle());
234                 jsonObject.put("url", link.getUrl());
235                 jsonObject.put("tags", new JSONArray(link.getTags()));
236                 jsonArray.put(jsonObject);
237             } catch (JSONException e) {
238                 Log.e("ExportUtil", "Error creating JSON object", e);
239             }
240         }
241         // ■ JSONObject ■■■■■■■■■■\ / -> /■■■ JSON ■■■
242         String jsonString = jsonArray.toString(4).replace("\\\\", "/");
243         writer.write(jsonString);
244     } else {
245         // ■■ CSV ■■
246         StringBuilder csv = new StringBuilder();
247         csv.append("■■,■■,■■,■■,■■■■,■■\n");
248
249         SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss", L
250

```

277 }


```

1  package person.notfresh.readingshare.util;
2
3  import android.content.ContentResolver;
4  import android.content.Context;
5  import android.net.Uri;
6  import android.util.Log;
7  import org.json.JSONArray;
8  import org.json.JSONObject;
9  import org.json.JSONException;
10 import person.notfresh.readingshare.model.LinkItem;
11
12 import java.io.BufferedReader;
13 import java.io.InputStream;
14 import java.io.InputStreamReader;
15 import java.text.ParseException;
16 import java.text.SimpleDateFormat;
17 import java.util.ArrayList;
18 import java.util.Date;
19 import java.util.List;
20
21 /**
22  * ██████████ CSV █ JSON ██████████
23  */
24 public class ImportUtil {
25
26     /**
27      * ████████
28      */
29     public static class ImportResult {
30         public final String format; // "CSV" █ "JSON"
31         public final int successCount;
32         public final int errorCount;
33         public final List<LinkItem> items;
34         public final List<String> errors;
35
36         public ImportResult(String format, int successCount, int errorCount,
37                             List<LinkItem> items, List<String> errors) {
38             this.format = format;
39             this.successCount = successCount;
40             this.errorCount = errorCount;
41             this.items = items;
42             this.errors = errors;
43         }
44

```